

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN
CÔNG NGHỆ PHẦN MỀM



Bài Tập Lớn
Kiểm Thử Phần Mềm
SHOPCART
(Version 1.1)

GVHD: Từ Lăng Phiêu
Môn học: Kiểm Thử Phần Mềm
Niên khóa: 2025 – 2026

TP. HỒ CHÍ MINH, THÁNG 04/2026

1 Giới thiệu về Dự án

1.1 Tổng quan

Dự án **ShopCart** là một ứng dụng web thương mại điện tử phục vụ cho việc kiểm thử các chức năng ở cả frontend và backend, gồm các nghiệp vụ sau:

- **Mua sản phẩm & Giỏ hàng (Cart):** Người dùng chọn mua, thêm sản phẩm vào giỏ hàng, cập nhật số lượng, xóa sản phẩm khỏi giỏ hàng.
- **Tính toán giá (Pricing):** Tính tổng giá trị đơn hàng hoặc giỏ hàng (đơn giá $\times$ số lượng), áp dụng mã giảm giá và cộng phí vận chuyển khi phát sinh.
- **Kiểm tra tồn kho (Inventory):** Xác nhận số lượng sản phẩm còn lại trong kho trước và sau thao tác; cảnh báo/khóa thao tác khi hết hàng hoặc vượt tồn.
- **Chức năng Mua Hàng (Purchase/Checkout):** Xác nhận đơn hàng, trừ tồn kho, tạo mã đơn hàng và trả về trạng thái xử lý.
- **Frontend:** React 19.x.
- **Backend:** Spring Boot 3.5.x hoặc 4.x, khởi tạo từ `start.spring.io`.
- **Kiểm thử phía Frontend:** **Vitest** (unit test) và **Playwright** (end-to-end).
- **Testing:** Khuyến khích áp dụng quy trình TDD cho các chức năng nghiệp vụ cốt lõi.

1.2 Công nghệ sử dụng

1.2.1 Frontend

- **React 19.x**
- **Vitest – Unit test** (tích hợp tốt với Vite)
- **React Testing Library** – Kiểm thử component React
- **Playwright – End-to-end (E2E)** trên trình duyệt
- **Axios** – HTTP client
- **Vite** – Build tool & Dev server
- **CSS3 / TailwindCSS** – Styling

1.2.2 Backend

- **Spring Boot** – Xây dựng REST API bằng Spring Boot 3.5.x hoặc 4.x
- **Java** – Sử dụng Java 21 hoặc phiên bản LTS tương thích với Spring Boot đã chọn
- **JUnit 5** – Testing framework
- **Mockito** – Mock framework
- **Maven** – Build tool
- **Spring Data JPA** – Database operations
- **Spring Security** – Bảo mật
- **H2 / PostgreSQL** – Database (test / production)

1.3 Cấu trúc dự án

```
ShopCart_FE_BE/
+-- frontend/                                -- React 19.x + Vite
|   +-- src/
|       +-- components/                    -- Cart, Checkout, Inventory components
|       +-- services/                     -- API services (cartService, orderService)
|       +-- utils/                        -- Validation, price calculation utilities
|       +-- hooks/                        -- Custom React hooks
|       +-- tests/                        -- Test files (Vitest)
|   +-- e2e/                              -- Playwright E2E test files
|   +-- playwright.config.ts
|   +-- vite.config.ts
|   +-- package.json
|
\-- backend/                                -- Spring Boot API
    +-- src/
        +-- main/java/com/shopcart/
            +-- controller/                -- CartController, OrderController
            |   |
            |   +-- service/                -- Business logic
            |   +-- dto/                   -- Data Transfer Objects
            |   +-- entity/                -- Database entities
            |   +-- repository/            -- Data access
            +-- test/java/                 -- Test files (JUnit 5 + Mockito)
        +-- pom.xml
```

2 Câu 1: Phân tích và Thiết kế Test Cases (0.5 điểm)

2.1 Câu 1.1: Giỏ Hàng (Cart) – Phân tích và Test Scenarios (0.25 điểm)

2.1.1 Yêu cầu (0.25 điểm)

Dựa trên chức năng Giỏ Hàng (Shopping Cart), hãy thực hiện:

a) Phân tích các yêu cầu chức năng chính của tính năng Cart (không tính điểm riêng, gộp vào mục b)

- Validation rules khi thêm sản phẩm vào giỏ hàng
- Validation rules cho số lượng sản phẩm
- Kiểm tra tồn kho trước khi thêm vào giỏ
- Tính toán tổng giá trị giỏ hàng
- Xử lý lỗi (sản phẩm hết hàng, số lượng vượt tồn kho)

b) Liệt kê, mô tả test scenarios và phân loại mức độ ưu tiên cho Cart (0.25 điểm):

- *Happy path*: Thêm sản phẩm hợp lệ vào giỏ hàng thành công
- *Negative tests*: Thêm sản phẩm hết hàng, số lượng âm, số lượng vượt tồn kho
- *Boundary tests*: Số lượng = 1 (min), số lượng = tồn kho tối đa
- *Edge cases*: Thêm cùng sản phẩm nhiều lần, xóa sản phẩm khỏi giỏ hàng trống

c) Phân loại test scenarios theo mức độ ưu tiên (Critical, High, Medium, Low) và giải thích (không tính điểm riêng, gộp vào mục b)

Validation Rules cho Cart:

- **Số lượng (Quantity)**: ≥ 1 , $\leq$ tồn kho hiện tại, kiểu số nguyên
- **Product ID**: Phải tồn tại trong hệ thống, không được null
- **Tồn kho**: Số lượng trong giỏ không được vượt quá tồn kho
- **Trạng thái sản phẩm**: Phải đang được bán (status = ACTIVE)

2.1.2 Thiết kế Test Cases chi tiết

Thiết kế 5 test cases quan trọng nhất cho chức năng Cart theo template:

Table 1: Template Test Case cho Cart – TC_CART_001

Trường	Nội dung
Test Case ID	TC_CART_001
Test Name	Thêm sản phẩm vào giỏ hàng thành công
Priority	Critical
Preconditions	- Người dùng đã đăng nhập - Sản phẩm tồn tại và còn hàng - Ứng dụng đang chạy
Test Steps	1. Truy cập trang danh sách sản phẩm 2. Chọn sản phẩm “Laptop Dell” 3. Nhập số lượng: 2 4. Nhấn nút “Thêm vào giỏ hàng”
Test Data	Product ID: P001 Product Name: Laptop Dell Quantity: 2 Stock available: 10
Expected Result	- Thông báo “Thêm vào giỏ hàng thành công” - Số lượng trong giỏ hàng tăng lên 2 - Icon giỏ hàng hiển thị badge = 2 - Tồn kho tạm thời giảm còn 8
Actual Result	(Để trống – sinh viên điền sau khi chạy test)
Status	Not Run

Sinh viên cần thiết kế thêm 4 test cases tương tự cho các scenarios: thêm quá tồn kho, sản phẩm hết hàng, xóa sản phẩm khỏi giỏ, cập nhật số lượng trong giỏ.

2.2 Câu 1.2: Mua Hàng (Purchase/Checkout) – Phân tích và Test Scenarios (0.25 điểm)

2.2.1 Yêu cầu (0.25 điểm)

Dựa trên chức năng Mua Hàng (Purchase & Checkout), hãy thực hiện:

a) Phân tích các yêu cầu chức năng chính của Purchase (không tính điểm riêng, gộp vào mục b)

- **Tính giá:** Tính tổng giá ($\text{unit price} \times \text{quantity}$), áp dụng giảm giá, phí vận chuyển
- **Kiểm tra tồn kho:** Xác nhận đủ hàng trước khi đặt
- **Đặt hàng:** Tạo order, trừ tồn kho, gửi xác nhận
- **Thanh toán:** Xác nhận phương thức thanh toán

b) Liệt kê, mô tả test scenarios và phân loại mức độ ưu tiên cho Purchase (0.25 điểm):

- *Happy path:* Đặt hàng thành công, tồn kho được cập nhật
- *Negative tests:* Đặt hàng khi hết hàng, giá trị đơn hàng âm

- *Boundary tests*: Giá trị đơn hàng tối thiểu, số lượng = 1
- *Edge cases*: Đặt hàng đúng số lượng tồn kho cuối cùng, mã giảm giá không hợp lệ

c) Phân loại test scenarios theo mức độ ưu tiên và giải thích (không tính điểm riêng, gộp vào mục b)

Validation Rules cho Purchase:

- **Tổng giá (Total Price)**: > 0 , tính chính xác = $\sum(\text{price} \times \text{qty})$
- **Mã giảm giá (Coupon)**: Còn hiệu lực, chưa sử dụng, đúng điều kiện áp dụng
- **Tồn kho**: Tất cả sản phẩm trong giỏ phải còn đủ hàng tại thời điểm checkout
- **Địa chỉ giao hàng**: Không được rỗng, hợp lệ

2.2.2 Thiết kế Test Cases chi tiết (không tính điểm riêng, dùng để hướng dẫn trình bày bài làm)

Table 2: Template Test Case cho Purchase – TC_PURCHASE_001

Trường	Nội dung
Test Case ID	TC_PURCHASE_001
Test Name	Đặt hàng thành công và tồn kho được cập nhật
Priority	Critical
Preconditions	- Người dùng đã đăng nhập - Giỏ hàng có ít nhất 1 sản phẩm - Tất cả sản phẩm còn đủ hàng
Test Steps	1. Truy cập trang Giỏ hàng 2. Xác nhận thông tin đơn hàng 3. Chọn phương thức thanh toán 4. Nhấn “Đặt hàng”
Test Data	Product: Laptop Dell (P001), qty=2, price=15.000.000đ Product: Mouse Logitech (P002), qty=1, price=500.000đ Coupon: SALE10 (giảm 10%) Shipping: 50.000đ
Expected Result	- Order tạo thành công (status: PENDING) - Tổng tiền = $(15.000.000 \times 2 + 500.000 \times 1) \times 0.9 + 50.000 = 27.950.000\text{đ}$ - Tồn kho P001 giảm từ 10 còn 8 - Tồn kho P002 giảm từ 50 còn 49 - Email xác nhận được gửi
Actual Result	(Để trống)
Status	Not Run

Sinh viên cần thiết kế thêm 4 test cases tương tự cho: đặt hàng khi hết hàng, kiểm tra tính toán giá sai, mã giảm giá hết hạn, tồn kho không đủ.

3 Câu 2: Unit Testing và Test-Driven Development (2 điểm)

3.1 Câu 2.1: Cart – Unit Tests Frontend và Backend (1 điểm)

3.1.1 Frontend Unit Tests – Cart Validation với Vitest (0.5 điểm)

Áp dụng TDD để phát triển unit tests cho validation module của Cart sử dụng **Vitest**:

Yêu cầu:

a) Viết unit tests cho validateCartItem() (0.25 điểm):

- Test quantity rỗng / null / undefined
- Test quantity âm hoặc bằng 0
- Test quantity vượt quá tồn kho
- Test quantity hợp lệ

b) Viết unit tests cho calculateCartTotal() (0.25 điểm):

- Test giỏ hàng rỗng
- Test tính tổng giá đúng với nhiều sản phẩm
- Test áp dụng mã giảm giá
- Test tổng giá sau khi xóa sản phẩm

c) Coverage $\geq 90\%$ cho cart validation module (không tính điểm riêng, dùng để đối chiếu chất lượng bài làm)

Ví dụ Frontend Unit Test với Vitest: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 import { describe, test, expect } from 'vitest';
2 import { validateCartItem } from './cartValidation';
3
4 describe('Cart Validation Tests', () => {
5   test('TC1: Quantity = 0 - nên trả về lỗi', () => {
6     const result = validateCartItem({ productId: 'P001',
7       quantity: 0, stock: 10 });
8     expect(result.error).toBe('Số lượng phải lớn hơn 0');
9   });
10 });

```

Listing 1: cartValidation.test.ts

3.1.2 Backend Unit Tests – CartService (0.5 điểm)

Viết unit tests cho CartService của backend với JUnit 5 và Mockito:

Yêu cầu:

a) Test method addToCart() với các scenarios (0.25 điểm):

- Thêm sản phẩm thành công
- Thêm sản phẩm đã có trong giỏ (cộng dồn số lượng)
- Thêm khi tồn kho không đủ
- Thêm sản phẩm không tồn tại

b) Test `removeFromCart()` và `updateQuantity()` (0.25 điểm)

c) Coverage $\geq 85\%$ cho `CartService` (không tính điểm riêng, dùng để đối chiếu chất lượng bài làm)

Ví dụ Backend Unit Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 @DisplayName("Cart Service Unit Tests")
2 @ExtendWith(MockitoExtension.class)
3 class CartServiceTest {
4
5     @Mock
6     private ProductRepository productRepository;
7
8     @Mock
9     private CartRepository cartRepository;
10
11     @InjectMocks
12     private CartService cartService;
13
14     @Test
15     @DisplayName("TC1: Thêm sản phẩm vào giỏ hàng thành công")
16     void testAddToCartSuccess() {
17         Product product = new Product(
18             "P001", "Laptop Dell", 15000000L, 10
19         );
20         CartItemRequest request = new CartItemRequest(
21             "P001", 2
22         );
23
24         when(productRepository.findById("P001"))
25             .thenReturn(Optional.of(product));
26         when(cartRepository.save(any(CartItem.class)))
27             .thenAnswer(inv -> inv.getArgument(0));
28
29         CartItem result = cartService.addToCart(
30             "user01", request
31         );
32
33         assertNotNull(result);
34         assertEquals(2, result.getQuantity());
35         assertEquals("Laptop Dell", result.getProductName());
36         verify(cartRepository, times(1))
37             .save(any(CartItem.class));
38     }
39 }

```

Listing 2: CartServiceTest.java

3.2 Câu 2.2: Mua Hàng (Purchase) – Unit Tests Frontend và Backend (1 điểm)

3.2.1 Frontend Unit Tests – Price Calculation với Vitest (0.5 điểm)

Áp dụng TDD cho tính toán giá và kiểm tra tồn kho:

Yêu cầu:

a) Viết unit tests cho calculateOrderPrice() (0.25 điểm):

- Test tính tổng giá trước giảm giá
- Test áp dụng coupon giảm % (ví dụ: 10%, 20%)
- Test áp dụng coupon giảm số tiền cố định
- Test tính phí vận chuyển
- Test tổng cuối cùng (subtotal + shipping - discount)

b) Viết tests cho checkInventoryAvailability() (0.25 điểm)

c) Coverage $\geq 90\%$ (không tính điểm riêng, dùng để đối chiếu chất lượng bài làm)

Ví dụ Price Calculation Test với Vitest: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```
1 import { describe, test, expect } from 'vitest';
2 import { calculateOrderPrice,
3       checkInventoryAvailability }
4       from './priceCalculation';
5
6 describe('Price Calculation Tests', () => {
7   test('TC1: Tính tổng giá không có giảm giá', () => {
8     const items = [
9       { price: 15000000, quantity: 2 },
10      { price: 500000, quantity: 1 },
11    ];
12    const result = calculateOrderPrice(
13      items, null, 50000
14    );
15
16    expect(result.subtotal).toBe(30500000);
17    expect(result.discount).toBe(0);
18    expect(result.shipping).toBe(50000);
19    expect(result.total).toBe(30550000);
20  });
21 });
```

Listing 3: priceCalculation.test.ts

3.2.2 Backend Unit Tests – OrderService (0.5 điểm)

Viết unit tests cho OrderService:

Yêu cầu:

a) Test CRUD operations cho Order (0.25 điểm):

- Test `createOrder()` – tạo đơn hàng, trừ tiền kho
- Test `getOrderById()` – lấy thông tin đơn hàng
- Test `cancelOrder()` – hủy đơn, hoàn tiền kho
- Test `calculateOrderTotal()` – tính tổng giá chính xác
- Test `checkStockBeforeOrder()` – kiểm tra tiền kho

b) Coverage $\geq 85\%$ cho OrderService (0.25 điểm)

Ví dụ Order Service Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 @DisplayName("Order Service Unit Tests")
2 @ExtendWith(MockitoExtension.class)
3 class OrderServiceTest {
4
5     @Mock
6     private OrderRepository orderRepository;
7
8     @Mock
9     private ProductRepository productRepository;
10
11     @Mock
12     private InventoryService inventoryService;
13
14     @InjectMocks
15     private OrderService orderService;
16
17     @Test
18     @DisplayName("TC1: Tao don hang thanh cong")
19     void testCreateOrderSuccess() {
20         OrderRequest request = OrderRequest.builder()
21             .userId("user01")
22             .items(List.of(
23                 new OrderItemRequest("P001", 2, 15000000L),
24                 new OrderItemRequest("P002", 1, 500000L)
25             ))
26             .couponCode("SALE10")
27             .shippingFee(50000L)
28             .build();
29
30         when(inventoryService.isAvailable("P001", 2))
31             .thenReturn(true);
32         when(inventoryService.isAvailable("P002", 1))
33             .thenReturn(true);
34         when(orderRepository.save(any(Order.class)))
35             .thenAnswer(inv -> {
36                 Order o = inv.getArgument(0);
37                 o.setId(UUID.randomUUID().toString());
38                 return o;
39             });
40
41         OrderResponse response = orderService
42             .createOrder(request);
43

```

```
44     assertNotNull(response.getId());
45     assertEquals(OrderStatus.PENDING,
46         response.getStatus());
47     assertEquals(27950000L, response.getTotalPrice());
48     verify(inventoryService, times(1))
49         .decreaseStock("P001", 2);
50     verify(inventoryService, times(1))
51         .decreaseStock("P002", 1);
52 }
53 }
```

Listing 4: OrderServiceTest.java

4 Câu 3: Integration Testing (2 điểm)

4.1 Câu 3.1: Cart – Integration Testing (1 điểm)

4.1.1 Frontend Component Integration với Vitest + React Testing Library (0.5 điểm)

Test tích hợp Cart component với API service:

Yêu cầu:

- a) Test rendering và user interactions (0.25 điểm)
- b) Test form submission và API calls (0.25 điểm)
- c) Test error handling và success messages (không tính điểm riêng, dùng để hoàn thiện bộ test)

Ví dụ Component Integration Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 import { describe, test, expect, vi } from 'vitest';
2 import { render, screen, fireEvent, waitFor }
3   from '@testing-library/react';
4 import CartComponent from '../CartComponent';
5 import * as cartService from '../services/cartService';
6
7 vi.mock('../services/cartService');
8
9 describe('Cart Component Integration Tests', () => {
10   test('Hiện thị giỏ hàng rỗng khi chưa có sản phẩm',
11     async () => {
12       vi.mocked(cartService.getCart)
13         .mockResolvedValue({ items: [], total: 0 });
14
15       render(<CartComponent userId="user01" />);
16
17       await waitFor(() => {
18         expect(screen.getByTestId('empty-cart-message'))
19           .toBeInTheDocument();
20       });
21     });
22 });

```

Listing 5: Cart.integration.test.tsx

4.1.2 Backend API Integration (0.5 điểm)

Test API endpoints của Cart với MockMvc:

Yêu cầu:

- a) Test POST /api/cart/add endpoint (0.25 điểm)
- b) Test response structure và status codes (0.25 điểm)
- c) Test CORS và headers (không tính điểm riêng, dùng để mở rộng bài làm)

Ví dụ API Integration Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 @WebMvcTest(CartController.class)
2 @DisplayName("Cart API Integration Tests")
3 class CartControllerIntegrationTest {
4
5     @Autowired
6     private MockMvc mockMvc;
7
8     @Autowired
9     private ObjectMapper objectMapper;
10
11     @MockBean
12     private CartService cartService;
13
14     @Test
15     @DisplayName("POST /api/cart/add - Them san pham")
16     void testAddToCart() throws Exception {
17         CartItemRequest request = CartItemRequest.builder()
18             .productId("P001").quantity(2).build();
19
20         CartResponse mockResponse = CartResponse.builder()
21             .success(true)
22             .message("Them vao gio hang thanh cong")
23             .cartTotal(300000000L)
24             .build();
25
26         when(cartService.addToCart(any(), any()))
27             .thenReturn(mockResponse);
28
29         mockMvc.perform(post("/api/cart/add")
30             .header("Authorization", "Bearer token123")
31             .contentType(MediaType.APPLICATION_JSON)
32             .content(objectMapper
33                 .writeValueAsString(request)))
34             .andExpect(status().isOk())
35             .andExpect(jsonPath("$.success").value(true))
36             .andExpect(jsonPath("$.message")
37                 .value("Them vao gio hang thanh cong"))
38             .andExpect(jsonPath("$.cartTotal")
39                 .value(300000000));
40     }
41 }

```

Listing 6: CartControllerIntegrationTest.java

4.2 Câu 3.2: Purchase – Integration Testing (1 điểm)

4.2.1 Frontend Component Integration (0.5 điểm)

Test tích hợp Checkout components:

Yêu cầu:

- Test CheckoutSummary component với dữ liệu giỏ hàng (0.25 điểm)
- Test PriceCalculator component (tính giá real-time) (0.25 điểm)
- Test InventoryWarning component (cảnh báo hết hàng) (không tính điểm riêng, dùng để mở rộng bài làm)

Ví dụ Checkout Component Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 import { describe, test, expect } from 'vitest';
2 import { render, screen, fireEvent, waitFor }
3   from '@testing-library/react';
4 import CheckoutPage from './CheckoutPage';
5
6 describe('Checkout Integration Tests', () => {
7   test('Hien thi tong gia chinh xac', async () => {
8     const mockCart = {
9       items: [
10        { name: 'Laptop Dell', price: 15000000,
11          quantity: 2 },
12        { name: 'Mouse Logitech', price: 500000,
13          quantity: 1 },
14      ]
15    };
16
17    render(<CheckoutPage cart={mockCart} />);
18
19    await waitFor(() => {
20      expect(
21        screen.getByTestId('subtotal-display')
22      ).toHaveTextContent('30.500.000');
23    });
24  });
25 });

```

Listing 7: Checkout.integration.test.tsx

4.2.2 Backend API Integration (0.5 điểm)

Test các API endpoints của Purchase/Order:

Yêu cầu:

- a) Test POST /api/orders (Tạo đơn hàng) (0.25 điểm)
- b) Test thêm 1 endpoint bất kỳ trong nhóm Order hoặc Inventory (0.25 điểm)
- c) Các endpoint còn lại được khuyến khích triển khai thêm để hoàn thiện bộ test

Ví dụ Order API Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 @WebMvcTest(OrderController.class)
2 @DisplayName("Order API Integration Tests")
3 class OrderControllerIntegrationTest {
4
5     @Autowired
6     private MockMvc mockMvc;
7
8     @MockBean
9     private OrderService orderService;
10
11     @MockBean

```

```
12 private InventoryService inventoryService;
13
14 @Test
15 @DisplayName("POST /api/orders - Tao don hang")
16 void testCreateOrder() throws Exception {
17     OrderRequest request = OrderRequest.builder()
18         .items(List.of(
19             new OrderItemRequest("P001", 2, 15000000L)))
20         .shippingFee(50000L)
21         .build();
22     OrderResponse mockResponse = OrderResponse.builder()
23         .orderId("ORD-001")
24         .status(OrderStatus.PENDING)
25         .totalPrice(30050000L)
26         .build();
27
28     when(orderService.createOrder(any()))
29         .thenReturn(mockResponse);
30
31     mockMvc.perform(post("/api/orders")
32         .contentType(MediaType.APPLICATION_JSON)
33         .content(objectMapper
34             .writeValueAsString(request)))
35         .andExpect(status().isCreated())
36         .andExpect(jsonPath("$.orderId")
37             .value("ORD-001"))
38         .andExpect(jsonPath("$.totalPrice")
39             .value(30050000));
40 }
41 }
```

Listing 8: OrderControllerIntegrationTest.java

5 Câu 4: Mock Testing (2 điểm)

5.1 Câu 4.1: Cart – Mock Testing (1 điểm)

5.1.1 Frontend Mocking với Vitest (0.5 điểm)

Mock external dependencies cho Cart component:

Yêu cầu:

- Mock `cartService.addToCart()` (0.25 điểm)
- Test với mocked successful/failed responses và verify mock calls (0.25 điểm)

Ví dụ Frontend Mock Test với Vitest: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 import { describe, test, expect, vi, beforeEach }
2   from 'vitest';
3 import { render, screen, fireEvent, waitFor }
4   from '@testing-library/react';
5 import CartComponent from '../CartComponent';
6 import * as cartService from '../services/cartService';
7
8 vi.mock('../services/cartService');
9
10 describe('Cart Mock Tests', () => {
11   beforeEach(() => {
12     vi.clearAllMocks();
13   });
14
15   test('Mock: Thêm sản phẩm thành công', async () => {
16     vi.mocked(cartService.addToCart)
17       .mockResolvedValue({
18         success: true,
19         message: 'Thêm vào giỏ hàng thành công',
20         cartTotal: 30000000
21       });
22
23     render(<CartComponent userId="user01" />);
24     fireEvent.click(
25       screen.getByTestId('add-to-cart-btn')
26     );
27
28     await waitFor(() => {
29       expect(cartService.addToCart)
30         .toHaveBeenCalledWith('user01',
31           expect.objectContaining({
32             productId: expect.any(String)
33           }));
34       expect(
35         screen.getByText(/thanh cong/i)
36       ).toBeInTheDocument();
37     });
38   });
39 });

```

Listing 9: Cart.mock.test.tsx

5.1.2 Backend Mocking (0.5 điểm)

Mock dependencies trong Backend tests:

Yêu cầu:

- a) Mock CartService với @MockBean (0.25 điểm)
- b) Test controller với mocked service (0.25 điểm)
- c) Verify mock interactions và kiểm tra phản hồi trả về từ controller khi service được mock (không tính điểm riêng, dùng để hoàn thiện bài làm)

Ví dụ Backend Mock Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 @WebMvcTest(CartController.class)
2 class CartControllerMockTest {
3
4     @Autowired
5     private MockMvc mockMvc;
6
7     @MockBean
8     private CartService cartService;
9
10    @Test
11    @DisplayName("Mock: Add to cart with mocked service")
12    void testAddToCartWithMockedService()
13        throws Exception {
14
15        CartResponse mockResponse = CartResponse.builder()
16            .success(true)
17            .message("Them vao gio hang thanh cong")
18            .cartTotal(300000000L)
19            .build();
20
21        when(cartService.addToCart(anyString(), any()))
22            .thenReturn(mockResponse);
23
24        mockMvc.perform(post("/api/cart/add")
25            .contentType(MediaType.APPLICATION_JSON)
26            .content("{\"productId\":\"P001\","
27                + "\"quantity\":2}"))
28            .andExpect(status().isOk())
29            .andExpect(jsonPath("$.success").value(true));
30
31        verify(cartService, times(1))
32            .addToCart(any(), any());
33    }
34 }

```

Listing 10: CartControllerMockTest.java

5.2 Câu 4.2: Purchase – Mock Testing (1 điểm)

5.2.1 Frontend Mocking với Vitest (0.5 điểm)

Mock OrderService trong component tests:

Yêu cầu:

- a) Mock orderService.createOrder(), inventoryService.checkStock() (0.25 điểm)

b) Test success và failure scenarios, đồng thời verify mock calls (0.25 điểm)

Ví dụ Purchase Mock Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 import { vi } from 'vitest';
2 import * as orderService from '../services/orderService';
3 import * as inventoryService
4   from '../services/inventoryService';
5
6 vi.mock('../services/orderService');
7 vi.mock('../services/inventoryService');
8
9 describe('Purchase Mock Tests', () => {
10   test('Mock: Dat hang thanh cong', async () => {
11     vi.mocked(inventoryService.checkStock)
12       .mockResolvedValue({ available: true });
13
14     vi.mocked(orderService.createOrder)
15       .mockResolvedValue({
16         orderId: 'ORD-001',
17         status: 'PENDING',
18         totalPrice: 30550000
19       });
20
21     // Test implementation
22     expect(orderService.createOrder)
23       .toHaveBeenCalledTimes(1);
24     expect(inventoryService.checkStock)
25       .toHaveBeenCalledWith(
26         expect.arrayContaining([
27           expect.objectContaining({ productId: 'P001' })
28         ])
29       );
30   });
31 });

```

Listing 11: Purchase.mock.test.tsx

5.2.2 Backend Mocking (0.5 điểm)

Mock InventoryRepository và OrderRepository trong service tests:

Yêu cầu:

- a) Mock InventoryRepository, OrderRepository (0.25 điểm)
- b) Test service layer với mocked repositories (0.25 điểm)
- c) Verify repository interactions và kiểm tra dữ liệu sau khi service xử lý với repository được mock (không tính điểm riêng, dùng để hoàn thiện bài làm)

Ví dụ Repository Mock Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 @ExtendWith(MockitoExtension.class)
2 class OrderServiceMockTest {
3
4   @Mock
5   private OrderRepository orderRepository;

```

```
6
7  @Mock
8  private ProductRepository productRepository;
9
10 @Mock
11 private InventoryRepository inventoryRepository;
12
13 @InjectMocks
14 private OrderService orderService;
15
16 @Test
17 void testCheckAndReduceInventory() {
18     Inventory inventory = new Inventory(
19         "P001", 10
20     );
21
22     when(inventoryRepository.findByProductId("P001"))
23         .thenReturn(Optional.of(inventory));
24     when(inventoryRepository.save(any()))
25         .thenAnswer(inv -> inv.getArgument(0));
26
27     orderService.checkAndReduceStock("P001", 3);
28
29     ArgumentCaptor<Inventory> captor =
30         ArgumentCaptor.forClass(Inventory.class);
31     verify(inventoryRepository).save(captor.capture());
32     assertEquals(7, captor.getValue().getQuantity());
33 }
34 }
```

Listing 12: OrderServiceMockTest.java

6 Câu 5: Automation Testing và CI/CD (2 điểm)

6.1 Câu 5.1: Cart – E2E Automation Testing với Playwright (1 điểm)

6.1.1 Setup và Configuration (0.25 điểm)

Yêu cầu:

- Cài đặt **Playwright** cho dự án React (bản mới nhất) + Vite
- Cấu hình `playwright.config.ts` (browsers: Chromium, Firefox, WebKit)
- Setup Page Object Model cho Cart pages

6.1.2 E2E Test Scenarios cho Cart (0.5 điểm)

Viết automated tests cho toàn bộ cart flow sử dụng Playwright:

Yêu cầu:

- a) Test complete add-to-cart flow (0.25 điểm)
- b) Test validation messages khi vượt tồn kho (0.25 điểm)
- c) Test success/error flows và UI elements interactions (không tính điểm riêng, dùng để hoàn thiện bài làm)

Ví dụ Playwright E2E Test: *Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.*

```
1 import { test, expect } from '@playwright/test';
2
3 test.describe('Cart E2E Tests', () => {
4   test.beforeEach(async ({ page }) => {
5     await page.goto('http://localhost:5173');
6     // Login trước khi test cart
7     await page.fill('[data-testid="username-input"]',
8       'testuser');
9     await page.fill('[data-testid="password-input"]',
10      'Test123');
11    await page.click('[data-testid="login-btn"]');
12    await page.waitForURL('**/products');
13  });
14
15  test('Thêm sản phẩm vào giỏ hàng thành công',
16    async ({ page }) => {
17    await page.fill(
18      '[data-testid="quantity-input"]', '2'
19    );
20    await page.click(
21      '[data-testid="add-to-cart-btn"]'
22    );
23
24    await expect(
25      page.locator('[data-testid="cart-badge"]')
26    ).toHaveText('2');
27    await expect(
28      page.locator('[data-testid="success-toast"]')
```

```
29     ).toContainText('them vao gio thanh cong');  
30   });  
31 }
```

Listing 13: cart.e2e.spec.ts

6.1.3 CI/CD Integration cho Cart Tests (0.25 điểm)

Yêu cầu:

- Tạo GitHub Actions workflow, chạy test tự động và lưu report (0.25 điểm)

Ví dụ CI/CD Config: Ví dụ dưới đây là cấu hình tham khảo; sinh viên cần điều chỉnh lại cho phù hợp với cấu trúc dự án của mình.

```
1 name: Cart Tests CI  
2  
3 on:  
4   push:  
5     branches: [main, develop]  
6   pull_request:  
7     branches: [main]  
8  
9 jobs:  
10  test-cart:  
11    runs-on: ubuntu-latest  
12    steps:  
13      - uses: actions/checkout@v4  
14  
15      - name: Setup Node.js  
16        uses: actions/setup-node@v4  
17        with:  
18          node-version: '22'  
19          cache: 'npm'  
20          cache-dependency-path: frontend/package-lock.json  
21  
22      - name: Install dependencies  
23        run: |  
24          cd frontend  
25          npm ci  
26  
27      - name: Run Cart Unit Tests (Vitest)  
28        run: |  
29          cd frontend  
30          npm run test -- --reporter=verbose \  
31            --testPathPattern=cart  
32  
33      - name: Install Playwright Browsers  
34        run: |  
35          cd frontend  
36          npx playwright install --with-deps  
37  
38      - name: Run Cart E2E Tests (Playwright)  
39        run: |  
40          cd frontend  
41          npx playwright test e2e/cart.e2e.spec.ts
```

```

42
43     - name: Upload Playwright Report
44       uses: actions/upload-artifact@v4
45       if: always()
46       with:
47         name: playwright-report
48         path: frontend/playwright-report/

```

Listing 14: .github/workflows/cart-tests.yml

6.2 Câu 5.2: Purchase – E2E Automation Testing với Playwright (1 điểm)

6.2.1 Setup Page Object Model (0.25 điểm)

Implement POM cho Checkout/Purchase pages với Playwright:

Ví dụ Page Object: Ví dụ dưới đây là cấu trúc tham khảo; sinh viên có thể điều chỉnh tên lớp, locator và phương thức theo giao diện thực tế.

```

1  import { Page, Locator, expect } from '@playwright/test';
2
3  export class CheckoutPage {
4      readonly page: Page;
5      readonly cartBadge: Locator;
6      readonly checkoutBtn: Locator;
7      readonly couponInput: Locator;
8      readonly applyCouponBtn: Locator;
9      readonly placeOrderBtn: Locator;
10     readonly totalDisplay: Locator;
11     readonly successMessage: Locator;
12     readonly inventoryWarning: Locator;
13
14     constructor(page: Page) {
15         this.page = page;
16         this.cartBadge =
17             page.locator('[data-testid="cart-badge"]');
18         this.checkoutBtn =
19             page.locator('[data-testid="checkout-btn"]');
20         this.couponInput =
21             page.locator('[data-testid="coupon-input"]');
22         this.applyCouponBtn =
23             page.locator('[data-testid="apply-coupon-btn"]');
24         this.placeOrderBtn =
25             page.locator('[data-testid="place-order-btn"]');
26         this.totalDisplay =
27             page.locator('[data-testid="total-display"]');
28         this.successMessage =
29             page.locator('[data-testid="order-success"]');
30         this.inventoryWarning =
31             page.locator('[data-testid="inventory-warning"]');
32     }
33
34     async goToCheckout() {
35         await this.checkoutBtn.click();
36         await this.page.waitForURL('**/checkout');

```

```

37 }
38
39 async applyCoupon(code: string) {
40   await this.couponInput.fill(code);
41   await this.applyCouponBtn.click();
42 }
43
44 async placeOrder() {
45   await this.placeOrderBtn.click();
46   await this.successMessage.waitFor();
47 }
48
49 async getTotalPrice(): Promise<string> {
50   return await this.totalDisplay.innerText();
51 }
52 }
53
54 export default CheckoutPage;

```

Listing 15: CheckoutPage.ts (Playwright POM)

6.2.2 E2E Test Scenarios cho Purchase (0.5 điểm)

Viết automated tests cho checkout flow:

Yêu cầu:

- a) Test complete checkout flow (0.25 điểm)
- b) Test tính giá chính xác (subtotal, discount, shipping) (0.25 điểm)
- c) Test áp dụng mã giảm giá, cảnh báo hết hàng hoặc cập nhật tồn kho sau khi đặt hàng (không tính điểm riêng, dùng để hoàn thiện bài làm)

Ví dụ Purchase E2E Test: Ví dụ dưới đây minh họa một test case đại diện; các test case còn lại sinh viên tự xây dựng theo yêu cầu.

```

1 import { test, expect } from '@playwright/test';
2 import CheckoutPage from '../pages/CheckoutPage';
3
4 test.describe('Purchase E2E Tests', () => {
5   let checkoutPage: CheckoutPage;
6
7   test.beforeEach(async ({ page }) => {
8     checkoutPage = new CheckoutPage(page);
9     // Setup: login va them san pham vao gio hang
10    await page.goto('http://localhost:5173');
11    await page.evaluate(() =>
12      localStorage.setItem('token', 'mock-token')
13    );
14    await page.goto('/cart');
15  });
16
17  test('Nen dat hang thanh cong va cap nhat ton kho',
18    async ({ page }) => {
19    await checkoutPage.goToCheckout();
20    await checkoutPage.placeOrder();
21
22    await expect(checkoutPage.successMessage)
23      .toContainText('dat hang thanh cong');

```

```

24     await expect(page)
25       .toHaveURL(/\/order-confirmation/);
26   });
27 });

```

Listing 16: purchase.e2e.spec.ts

6.2.3 CI/CD Integration – Complete Pipeline (0.25 điểm)

Setup complete CI/CD pipeline cho toàn bộ dự án, gồm các thành phần sau:

- Tự động chạy backend tests, frontend tests, Playwright E2E tests và lưu report (0.25 điểm)

Ví dụ CI/CD Pipeline hoàn chỉnh: *Ví dụ dưới đây là cấu hình tham khảo; sinh viên cần điều chỉnh lại cho phù hợp với cấu trúc dự án của mình.*

```

1 name: Complete CI/CD Pipeline
2
3 on:
4   push:
5     branches: [main]
6
7 jobs:
8   backend-tests:
9     runs-on: ubuntu-latest
10    services:
11      postgres:
12        image: postgres:17
13        env:
14          POSTGRES_DB: testdb
15          POSTGRES_PASSWORD: password
16        ports:
17          - 5432:5432
18    steps:
19      - uses: actions/checkout@v4
20
21      - name: Setup Java 21
22        uses: actions/setup-java@v4
23        with:
24          java-version: '21'
25          distribution: 'temurin'
26
27      - name: Backend Tests (JUnit 5 + Mockito)
28        run: |
29          cd backend
30          ./mvnw clean test jacoco:report
31
32      - name: Upload Backend Coverage
33        uses: codecov/codecov-action@v4
34        with:
35          files: ./backend/target/site/jacoco/jacoco.xml
36
37   frontend-tests:
38     runs-on: ubuntu-latest
39     steps:
40       - uses: actions/checkout@v4

```

```
41
42   - name: Setup Node.js 22
43     uses: actions/setup-node@v4
44     with:
45       node-version: '22'
46
47   - name: Install dependencies
48     run: |
49       cd frontend
50       npm ci
51
52   - name: Run Unit Tests (Vitest)
53     run: |
54       cd frontend
55       npm run test -- --coverage
56
57   - name: Upload Frontend Coverage
58     uses: codecov/codecov-action@v4
59     with:
60       files: ./frontend/coverage/lcov.info
61
62 e2e-tests:
63   runs-on: ubuntu-latest
64   needs: [backend-tests, frontend-tests]
65   steps:
66     - uses: actions/checkout@v4
67
68     - name: Setup Node.js 22
69       uses: actions/setup-node@v4
70       with:
71         node-version: '22'
72
73     - name: Install dependencies
74       run: |
75         cd frontend && npm ci
76
77     - name: Install Playwright
78       run: |
79         cd frontend
80         npx playwright install --with-deps
81
82     - name: Run E2E Tests (Playwright)
83       run: |
84         cd frontend
85         npx playwright test
86
87     - name: Upload Playwright Report
88       uses: actions/upload-artifact@v4
89       if: always()
90       with:
91         name: playwright-html-report
92         path: frontend/playwright-report/
93         retention-days: 30
```

Listing 17: .github/workflows/ci.yml

7 Câu 6: Advanced Testing – Performance và Security (1.5 điểm)

7.1 Câu 6.1: Performance Testing (0.75 điểm)

Đánh giá hiệu năng của các chức năng Cart và Checkout bằng công cụ như **k6** hoặc **JMeter**.

Yêu cầu:

- a) Xây dựng kịch bản tải cho ít nhất 1 API của Cart hoặc Checkout, nêu rõ số lượng người dùng đồng thời, thời lượng chạy test và các chỉ số cần đo (0.25 điểm)
- b) Thực thi performance test và thu thập các chỉ số chính như response time, throughput, error rate (0.25 điểm)
- c) Phân tích kết quả, xác định điểm nghẽn và đề xuất hướng tối ưu (0.25 điểm)

7.2 Câu 6.2: Security Testing (0.75 điểm)

Thực hiện kiểm thử bảo mật cho các luồng Cart, Checkout hoặc API liên quan.

Yêu cầu:

- a) Thiết kế test cases cho ít nhất 2 nhóm rủi ro như SQL Injection, XSS, IDOR, CSRF hoặc kiểm tra phân quyền truy cập API (0.25 điểm)
- b) Thực thi test hoặc mô phỏng request kiểm thử và ghi nhận kết quả (0.25 điểm)
- c) Mô tả tác động bảo mật và đề xuất biện pháp khắc phục (0.25 điểm)

8 Tiêu chí Đánh giá

8.1 Bảng Phân bổ Điểm

Table 3: Bảng phân bổ điểm chi tiết

Nội dung	Cart	Purchase	Tổng
Câu 1: Phân tích và Test Cases	0.25	0.25	0.5
Câu 2: Unit Testing và TDD	1.0	1.0	2.0
Câu 3: Integration Testing	1.0	1.0	2.0
Câu 4: Mock Testing	1.0	1.0	2.0
Câu 5: Automation (Playwright) & CI/CD	1.0	1.0	2.0
Câu 6: Advanced Testing (Performance & Security)	0.75	0.75	1.5
Tổng điểm bắt buộc	5.0	5.0	10.0
Điểm tối đa	10.0		

8.2 Tiêu chí Chất lượng

8.2.1 Code Quality (30%)

- Clean code principles
- Proper test structure (AAA pattern: Arrange – Act – Assert)
- Meaningful test names (mô tả rõ scenario và expected behavior)
- Test coverage $\geq 80\%$ (Vitest coverage cho frontend, JaCoCo cho backend)
- All tests pass (green build)

8.2.2 Documentation (20%)

- Test cases trình bày theo đúng template và mô tả rõ tiền điều kiện, bước thực hiện, kết quả mong đợi
- Ảnh chụp màn hình và minh chứng kết quả chạy test
- Test reports (Vitest HTML report, Playwright HTML report, JaCoCo)
- README với hướng dẫn cài đặt và chạy tests

8.2.3 Completeness (30%)

- Hoàn thành tất cả yêu cầu bắt buộc (Câu 1 đến Câu 6)
- Có test cases cho cả Cart và Purchase/Inventory
- CI/CD pipeline hoạt động (GitHub Actions xanh)
- Playwright E2E tests chạy thành công trên ít nhất 2 browsers

8.2.4 Best Practices (20%)

- Áp dụng quy trình TDD theo ba bước Red → Green → Refactor
- Mocking strategy hợp lý (chỉ mock dependency bên ngoài phạm vi cần kiểm thử)
- Good test data management (fixtures, factories)
- Tổ chức Playwright Page Object Model đúng giữa các màn hình kiểm thử
- Cấu hình Vitest phù hợp với dự án (ví dụ: concurrent tests, watch mode)

9 Hướng dẫn Nộp bài

9.1 Format Nộp bài

9.1.1 Source Code

- Git repository (GitHub/GitLab) – Public.
- Commit history rõ ràng (tuân thủ Conventional Commits)
- README.md có đủ hướng dẫn cài đặt, chạy ứng dụng, chạy test và xem report
- Có .gitignore để loại trừ node_modules, target, .env và các file build tạm thời
- Cấu trúc thư mục bám theo template đã cung cấp

9.1.2 Báo cáo

- File PDF (tối đa 20 trang)
- Đặt tên file: Test_Report_YourName.pdf
- **MỖI SINH VIÊN NỘP MỘT BÁO CÁO RIÊNG BIỆT, KHÔNG CHẤP NHẬN BÁO CÁO CHUNG**
- Bao gồm:
 - Giới thiệu project và công nghệ sử dụng
 - Test cases chi tiết (Cart + Purchase/Inventory)
 - Test execution results (screenshots)
 - Vitest coverage report (frontend unit tests)
 - Playwright HTML report (E2E tests)
 - JaCoCo coverage report (backend unit tests)
 - CI/CD pipeline documentation (GitHub Actions screenshots)
 - Kết luận và bài học kinh nghiệm

9.2 Thời hạn

DEADLINE: 23h59, ngày 10/05/2026

Thời gian làm bài: **4 tuần**

Không chấp nhận nộp trễ.

9.3 Demo (Optional)

- Video demo 10–15 phút
- Demo chạy Vitest unit tests (coverage report)
- Demo chạy Playwright E2E tests (headed mode, HTML report)
- Demo CI/CD pipeline (GitHub Actions)
- Q&A với giảng viên

References

- [1] React Documentation, <https://react.dev>, tài liệu chính thức cho React và hướng dẫn kiểm thử.
- [2] Vitest Documentation, <https://vitest.dev>, Unit testing framework cho Vite.
- [3] Playwright Documentation, <https://playwright.dev>, Kiểm thử end-to-end.
- [4] Spring Boot Documentation, <https://spring.io/projects/spring-boot>, tài liệu tham khảo chính thức cho Spring Boot.
- [5] JUnit 5 User Guide, <https://junit.org/junit5/>, Testing Framework for Java.
- [6] Mockito Framework, <https://site.mockito.org/>, Mocking Framework for Java.
- [7] React Testing Library, <https://testing-library.com/docs/react-testing-library/intro/>, Testing utilities for React.
- [8] Test-Driven Development: By Example, Kent Beck, Addison-Wesley Professional.
- [9] k6 Performance Testing, <https://k6.io/docs/>, Modern load testing tool for developers.
- [10] GitHub Actions Documentation, <https://docs.github.com/en/actions>, CI/CD with GitHub Actions.